Refer to “OPTIONAL” section at the end of pdf for better understanding

UNIT 1 - Introduction to Blockchain and distributed ledgers

Blockchain

- Blockchain is a **distributed and decentralized digital ledger** used to record transactions in a secure and immutable manner.
- Transactions are stored in **blocks**, and each block is linked to the previous block using cryptographic techniques, forming a **chain of blocks**.
- Once data is recorded in a blockchain, it **cannot be altered or deleted**, which ensures **data integrity and trust** without relying on a central authority.
- Blockchain is a decentralized and distributed ledger technology that stores transactions in blocks linked using cryptography to ensure security, transparency, and immutability.

Distributed Ledger

- A **distributed ledger** is a database that is **shared and synchronized across multiple computers (nodes)** in a network.
- Each participant maintains a **copy of the same ledger**, and any update is reflected across all copies after verification.
- Blockchain is a **type of distributed ledger**, but not all distributed ledgers are blockchains.

Key Features of Blockchain

1. Decentralized

Blockchain **does not rely on a central authority**. Control of data is shared among all participating nodes, which **removes dependency** on a single system and avoids single point of failure.
Example: Unlike banks controlling transaction records, blockchain records are maintained by the network itself.

---

2. Distributed

Blockchain data is **stored and synchronized across multiple computers** (nodes). Each node has a copy of the ledger, which **ensures data availability and reliability.**
Example: Even if one node fails, other nodes still retain the complete data.

---

3. Consensus (agreement)

Before adding a transaction, blockchain **requires agreement among network participants** using predefined rules. This ensures only valid transactions are recorded.
Example: A transaction is added only after most nodes confirm it is correct.

---

4. Unanimous (majority)

Transactions are accepted only after collective approval by network participants, ensuring fairness and trust in the system.
Example: No single node can force a false transaction into the blockchain.

---

5. Immutable (can’t modify)

Once data is recorded in a blockchain, it cannot be altered or deleted. This maintains data integrity and prevents tampering.
Example: Past **transaction records remain permanent and unchanged**.

---

6. Secure

Blockchain uses cryptographic **techniques such as hashing and encryption** to protect data from unauthorized access.
Example: Any attempt to modify a block is easily detected due to hash mismatch.

---

7. Faster Settlement

Blockchain **enables faster transaction processing** by eliminating intermediaries and manual verification processes.
Example: Blockchain-based transfers are completed faster than traditional bank transfers.

Centralized, Distributed, and Decentralized are **general system architecture concepts**. They are used to describe **how control and data are organized** in *any* system — not just blockchain.

Blockchain = Decentralized + Distributed system

Aspect	Centralized System	Distributed System	Decentralized System
Network / Hardware Resources	Owned and controlled by a single entity at one central location	Spread across multiple locations and data centers, usually owned by a service provider	Owned and shared by network members; no single owner
Solution Components	Maintained and managed by a central authority	Maintained and controlled by the solution provider	Each participant maintains an identical copy of the ledger
Data Ownership	Data is stored, maintained, and controlled by one central entity	Data is generally owned and managed by the customer	Data is added only after group consensus
Control	Full control lies with the central authority	Control is shared between service provider and customer	No single authority; control is shared by all participants
Single Point of Failure	Yes	No	No
Fault Tolerance	Low	High	Extremely high
Security Responsibility	Security handled by the central authority	Security is a shared responsibility	Security increases as more participants join
Performance	Controlled and optimized by central authority	Improves as infrastructure scales	May decrease as number of participants increases
Example	ERP system	Cloud computing	Blockchain

1.2 Application, limits, and challenges of Blockchain.

A. Applications of Blockchain

Blockchain is used where **trust, security, and transparency** are important.

1. Cryptocurrency

Blockchain is used to **record and verify digital currency transactions without banks or intermediaries.**
Example: Sending digital money directly from one person to another.

---

2. Banking and Financial Services

Blockchain enables faster, secure, and low-cost transactions, especially for international transfers.
Example: Cross-border payments with reduced processing time.

---

3. Supply Chain Management

Blockchain **tracks products from origin to final consumer**, ensuring transparency and authenticity.
Example: Tracking food products from farm to store.

---

4. Healthcare

Patient records can be securely stored and shared among hospitals with proper authorization.
Example: Doctors accessing verified medical history.

---

5. Digital Identity

Blockchain helps create secure and tamper-proof digital identities.
Example: Online identity verification without centralized databases.

---

6. Voting Systems

Blockchain can be used to conduct secure and transparent electronic voting.
Example: Preventing vote tampering in online elections.

B. Limitations of Blockchain

1. Scalability (more users -> low performance**)**

Blockchain networks process transactions slowly as the number of users increases.
Example: High transaction delay during heavy network usage.

---

2. High Energy Consumption

Some blockchains consume large amounts of energy for transaction validation.
Example: Continuous operation of mining systems.

---

3. Storage Requirements

Each node stores a copy of the entire blockchain, requiring large storage space.
Example: Increasing ledger size over time.

---

4. Irreversibility

Once data is added, it cannot be modified or deleted.
Example: Mistaken transactions cannot be corrected easily.

C. Challenges of Blockchain

1. Regulatory and Legal Issues

Lack of clear laws and regulations for blockchain usage in many countries.
Example: Legal uncertainty in cryptocurrency transactions.

---

2. Integration with Existing Systems

Difficulty in integrating blockchain with traditional systems.
Example: Banks migrating from legacy systems.

---

3. Privacy Concerns

Public blockchains **expose transaction data**, which may affect privacy.
Example: Transaction details visible to network participants.

---

4. Lack of Skilled Professionals

Blockchain is still new, and skilled developers are limited.

1.3 Basics of Cryptography

Cryptography

Cryptography is the method of **protecting information** by converting readable data (plaintext) into an **unreadable format (ciphertext)** so that only authorized users can access it. It is used to provide **confidentiality, integrity, authentication, and non-repudiation**.

Goals of Cryptography

- **Confidentiality** – Data is hidden from unauthorized users
- **Integrity** – Data cannot be altered without detection
- **Authentication** – Verifies the identity of users
- **Non-repudiation** – Sender cannot deny sending the message

---

Cryptographic Techniques

---

1. Private Key Cryptography (Secret Key Technique)

Private key cryptography uses **a single shared key** for both encryption and decryption.

The key must be kept secret between sender and receiver.

It is **fast and efficient**, but the major challenge is **securely sharing the key**.

Example: If A and B share the same secret password, A encrypts the message using the password and B decrypts it using the same password.

Used for: File encryption, Secure storage

---

2. Public Key Cryptography (Asymmetric Key Technique)

Public key cryptography uses **two keys**:

- **Public key** (shared openly)
- **Private key** (kept secret)

Data encrypted using a public key can only be decrypted using the corresponding private key. This avoids the need to share secret keys.

Example: Anyone can send an encrypted message using the receiver's public key, but only the receiver can read it using their private key.

Used for: Secure communication, Digital signatures

Types of Cryptography

---

1. Symmetric Cryptography

Symmetric cryptography uses **one common key** for encryption and decryption.
It is simple and fast but less secure if the key is exposed.

Advantages:

- High speed
- Simple implementation

Disadvantages: Key distribution problem

Example: Password-protected ZIP file.

---

2. Asymmetric Cryptography

Asymmetric cryptography uses **a pair of keys**—public and private.
It provides higher security and supports digital signatures.

Advantages:

- Secure key exchange
- Better authentication

Disadvantages: Slower than symmetric cryptography

Example: Secure email communication.

---

3. Hash Cryptography

Hash cryptography converts data into a **fixed-length hash value** using a hash function.
It is a **one-way process**, so original data cannot be retrieved from the hash.
Hashing is mainly used for **data integrity and verification**, not encryption.

Characteristics:

- Same input → same hash
- Small change → completely different hash
- Irreversible

Example: Password storage and blockchain block hashing.

Digital Signature Schemes

A **digital signature scheme** is a cryptographic method used to **verify the authenticity, integrity, and ownership** of a digital message or transaction.

It proves:

- **Who sent the message** (authentication)
- **That the message was not changed** (integrity)
- **Sender cannot deny sending it** (non-repudiation)

How a Digital Signature Works

A digital signature scheme uses **public key cryptography**. (reversed)

Steps:

1. Sender creates a **hash** of the message
2. Hash is encrypted using the **sender's private key** → this is the **digital signature**
3. Receiver decrypts the signature using the **sender's public key**
4. If hashes match → message is **authentic and unchanged**

Example: Signed PDF or Online Form

- You fill an online form and submit it.
- The system attaches a digital signature to confirm:
  - Who submitted it
  - That the form was not changed later
- If even one word is modified, the signature becomes invalid.

Digital signature ensures authenticity + integrity.

Use of Digital Signatures in Blockchain

- Verifying transaction sender
- Preventing fake transactions
- Proving ownership of digital assets

Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography (ECC) is a **public key cryptography technique** that uses **elliptic curve mathematics** to provide strong security with **smaller key sizes**.

ECC offers the **same level of security as traditional methods**, but with **less computation and storage**.

ECC is based on **points on an elliptic curve** defined by a mathematical equation. Operations on these points are **easy to perform**, but **extremely hard to reverse**, which provides security.

Why ECC is Important

- High security
- Smaller keys
- Faster computation
- Lower power consumption

Example

- ECC uses a **256-bit key**
- Traditional methods may need **2048-bit key** for same security

---

Role of ECC in Blockchain

- Used to generate **public and private keys**
- Used in **digital signature schemes**
- Makes blockchain secure and efficient

1.4 Consistency, Availability, and Partition Tolerance in Blockchain

---

CAP Theorem

- CAP theorem is also known as **Brewer's Theorem**
- Proposed by **Eric Brewer** in 1998 (proved later by Gilbert and Lynch)
- States that a **distributed system cannot guarantee all three** properties at the same time:
  - Consistency (C)
  - Availability (A)
  - Partition Tolerance (P)
- A system can achieve **at most two out of three** CAP properties simultaneously
- CAP principles are very important in **distributed systems**, especially blockchain

Consistency

- Ensures that **all nodes see the same data at the same time**
- After a write operation, all future read operations return the **updated data**
- Prevents different versions of data across nodes
- Strong consistency requires **coordination among nodes**
- This coordination can **increase latency** and affect performance

Example:

After a transaction is confirmed, every node shows the same balance.

Availability

- Ensures that **every request receives a response**
- Response is guaranteed, but it may **not contain the latest data**
- High availability is important for systems that need **continuous service**
- Node failures should not stop the system

Example:

Users can access blockchain services even if some nodes are down.

Partition Tolerance

- Ensures the system continues working **even when network partitions occur**
- Network partition happens when **communication between nodes is disrupted**
- System may split into independent sub-networks
- Each sub-network continues functioning until communication is restored

Example:

Blockchain nodes continue processing even if internet connectivity fails between them.

UNIT 2 - Structure of Blockchain

2.1 Types of Blockchain

Blockchain systems can be classified based on **access control, permission model, and use of tokens**.

Based on access control

1. Public Blockchain

- A public blockchain is **open to everyone** and anyone can join the network.
- Any user can **read data, send transactions, and participate in validation**.
- It is fully **decentralized** and does not require trust in a single authority.
- Public blockchains offer **high transparency and security**, but **scalability is limited**.

Example: Cryptocurrency networks where anyone can participate.

---

2. Private Blockchain

- A private blockchain is **controlled by a single organization**.
- Only authorized users are allowed to **read, write, or validate transactions**.
- It provides **better performance and privacy** compared to public blockchain.
- Used mainly for **internal organizational purposes**.

Example: A company using blockchain to manage internal records.

Based on permission model

1. Permissioned Blockchain

- In a permissioned blockchain, **participants must be authorized** before joining.
- Roles such as reading, writing, and validation are **restricted and controlled**.
- It offers **better security, compliance, and access control**.
- Can be implemented as **public or private** blockchain.

Example: Banking consortium blockchain where only approved banks participate.

---

2. Permissionless Blockchain

- In a permissionless blockchain, **no permission is required** to join the network.
- Anyone can become a node and participate in **transaction validation**.

- Ensures **openness and decentralization**, but may suffer from lower speed.
- Commonly used in **public blockchain systems**.

Example: Open cryptocurrency networks.

Based on use of tokens

1. Tokenized Blockchain

- A tokenized blockchain uses **digital tokens** to represent value or assets.
- Tokens are used for **transactions, incentives, and network participation**.
- Widely used in **cryptocurrency and decentralized applications**.

Example: Blockchains where tokens are used for payments or rewards.

---

2. Tokenless Blockchain

- A tokenless blockchain does **not use any digital currency or token**.
- It focuses on **data sharing, record management, and process automation**.
- Mainly used in **enterprise and business applications**.

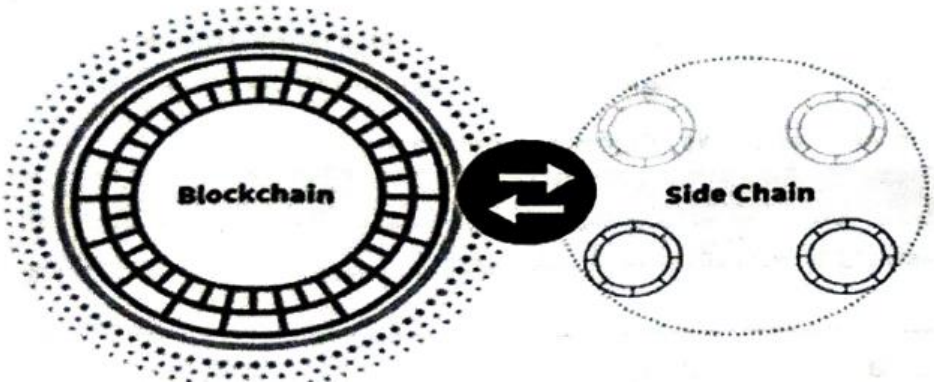
Example: Blockchain used for supply chain tracking without cryptocurrency

Point	Public Blockchain	Private Blockchain
Access	Open to everyone; anyone can join and participate	Access is restricted to authorized users only
Control	No single authority controls the network	Controlled by a single organization
Transparency	Fully transparent; all transactions are visible to participants	Limited transparency; data visibility is controlled
Performance	Slower due to large number of participants and consensus	Faster due to fewer nodes and controlled access
Use case	Used in open systems like cryptocurrencies	Used in enterprise and internal organizational systems

Point	Permissioned Blockchain	Permissionless Blockchain
Entry	Users need approval to join the network	Anyone can join without permission
Identity	Participants are known and verified	Participants can remain anonymous
Control	Access rights are defined and managed	No central authority manages access
Security	High security through access control	Security through cryptography and consensus
Usage	Suitable for business and consortium networks	Suitable for open public networks

Point	Tokenized Blockchain	Tokenless Blockchain
Use of tokens	Uses digital tokens or cryptocurrency	Does not use any digital tokens
Purpose	Tokens represent value, assets, or incentives	Focuses on data sharing and record management
Incentives	Participants are rewarded using tokens	No financial incentive mechanism
Complexity	More complex due to token economics	Simpler design and operation
Application	Used in cryptocurrencies and decentralized apps	Used mainly in enterprise applications

2.2 Sidechain



- A **sidechain** is an independent blockchain that runs **parallel to a main blockchain**.
- It is connected to the main chain through a **two-way mechanism**, allowing assets or data to move between them.
- Sidechains help extend the functionality of the main blockchain **without overloading it**.

1. Interoperability

- Both chains are interoperable, meaning they can **communicate and exchange assets**.
- This allows users to move assets between the main chain and the sidechain smoothly.

---

2. Scalability

- One of the main purposes of sidechains is to **improve scalability**.
- This reduces congestion and helps **improve overall network performance**.

Example: High-volume transactions are processed on a sidechain instead of the main chain.

---

3. Customization and Specialization

- Sidechains can be designed with **specific features tailored to particular use cases**.
- They can be optimized for faster confirmation times, lower fees, or enhanced privacy.
- This allows industries to use blockchain according to their **specific requirements**.

Example: A sidechain optimized for fast and low-cost transactions in gaming applications.

---

4. Two-Way Peg

- Asset transfer between the main chain and sidechain is enabled through a **two-way peg mechanism**.
- Assets are **locked on the main chain** when moved to the sidechain and unlocked when returned.
- This ensures asset value is **preserved and prevents duplication**.

Example: Coins are locked on the main blockchain when used on the sidechain and released back later.

---

5. Security

- **Cryptographic proofs and validation** mechanisms are used to secure transactions.
- Security ensures integrity and trust in sidechain operations.

Example: Cryptographic verification prevents unauthorized modification of sidechain transactions.

Examples of Sidechain

- **Bitcoin Sidechains**: Used to test new features like faster transactions and smart contracts without changing Bitcoin's core protocol.
- **Ethereum Sidechains**: Used for decentralized applications that require high speed and low transaction cost.
- **Gaming Sidechains**: Handle frequent in-game transactions without congesting the main blockchain.

2.3 Core Components of Blockchain

1. Node

- A node is a **computer or device** that participates in the blockchain network.
- Nodes store a copy of the blockchain and help in **verifying and propagating transactions**.
- They maintain decentralization by ensuring no single authority controls the network.

Example: A computer running blockchain software and maintaining the ledger.

---

2. Transaction

- A transaction represents a **record of an action or exchange** on the blockchain.
- It contains details such as sender, receiver, amount, and digital signature.
- Transactions are verified before being added to a block.
- Once confirmed, transactions become **permanent and immutable**.

Example: Sending digital assets from one user to another.

---

3. Block

- A block is a **data structure** that stores a group of verified transactions.
- It contains transaction data, timestamp, hash of the previous block, and its own hash.
- Blocks ensure data is stored in an **organized and secure manner**.

Example: A **container holding multiple transactions** recorded at a specific time.

---

4. Chain

- The chain is a **linked sequence of blocks** arranged chronologically.
- Each block is connected to the previous block using cryptographic hash values.
- Any change in one block breaks the entire chain.

Example: A continuous sequence of blocks forming the blockchain ledger.

---

5. Miners

- Miners are special nodes that **validate transactions and create new blocks**.
- **They compete to solve complex mathematical problems** to add blocks.
- Mining helps secure the network and maintain trust.

Example: Nodes validating transactions and adding new blocks to the blockchain.

---

6. Consensus

- Consensus is the mechanism through which **all nodes agree on a valid state of the blockchain**.
- It ensures that only valid transactions are added to the chain.
- Consensus **prevents fraud** and double spending.
- Different blockchains use different consensus methods.

Example: Nodes agreeing before a new block is added to the blockchain.

2.4 Distributed identity

Distributed identity is a digital identity system in which an individual owns and controls their identity information without relying on a central authority. It's Components are:

Public and Private Keys

- Public and private keys are **cryptographic keys** used to identify and authenticate users in a blockchain system.
- The **public key** is shared openly and acts as a digital identity or address.
- The **private key** is kept secret and is used to prove ownership and authorize actions.
- Together, they enable secure authentication without revealing personal information.

Example:
A user signs a transaction using a private key, and others verify it using the public key.

Digital Identification

- Digital identification is the process of **representing a user's identity in digital form**.
- In blockchain, identity is verified using cryptographic proofs instead of usernames and passwords.
- It reduces dependency on centralized identity providers and prevents identity theft.
- Users have full control over when and how their identity information is shared.

Example: Verifying identity for online services using cryptographic credentials instead of ID cards.

Wallets

- A wallet is a **software or hardware tool** used to store and manage public and private keys.
- Wallets allow users to **send, receive, and manage digital assets and identity credentials**.
- They do not store actual assets but store the keys that give access to them.
- Wallets play a key role in managing distributed identity securely.

Example: A digital wallet used to manage blockchain identity and authorize transactions.

Importance of Distributed Identity

- Eliminates reliance on centralized identity systems
- Enhances privacy and user control
- Reduces identity fraud and data breaches
- Enables secure authentication in decentralized applications

---

2.5 Decentralized Network and Distributed Ledger

Decentralized Network

- A decentralized network is a network where **no single central authority controls the system**.
- Control and decision-making are **shared among multiple nodes** participating in the network.
- Each node can independently verify transactions, increasing **trust and reliability**.
- The failure of one or a few nodes **does not affect the entire network**.
- Decentralization removes single point of failure and reduces the risk of manipulation.

Example: Blockchain networks where transactions are validated collectively by nodes instead of a central server.

Advantages

- **Removes central authority**, **reducing dependency** on a single controlling entity.
- **Improves reliability** since failure of one node does not stop the network.
- **Increases transparency** through participation of multiple nodes in verification.
- **Reduces risk of data manipulation** and censorship.
- **Builds trust** by distributing control among participants.

Disadvantages

- **Decision-making is slower** due to involvement of many nodes.
- **Requires complex consensus** mechanisms to maintain agreement.
- **Large networks are difficult to manage** and coordinate.
- **Performance may reduce** as the number of participants increases.

Distributed Ledger

- A distributed ledger is a **shared database** that is maintained across **multiple nodes** in a network.
- Every participating node stores an **identical copy of the ledger**.
- Any update to the ledger is recorded only after **network verification or consensus**.
- Distributed ledgers ensure **data transparency, integrity, and fault tolerance**.
- Blockchain is a **type of distributed ledger**, but not all distributed ledgers are blockchains.

Example: All blockchain nodes maintaining the same transaction history.

Advantages

- **Ensures high availability** as data is stored on multiple nodes.
- **Improves fault tolerance** and overall system reliability.
- **Provides transparency** since all participants share the same data.
- **Maintains data integrity** through synchronization and verification.
- **Reduces reliance on centralized databases**.

Disadvantages

- **Requires large storage** as the ledger grows continuously.
- Data synchronization can introduce delays.
- **Managing updates and consistency is technically complex.**
- Integration with existing systems can be challenging.

2.6 Data Structure of a Blockchain

---

1. Block

- A block is the basic data unit of a blockchain.
- It stores transaction data along with metadata required for security and linking.
- Blocks are added in a sequential order, forming a chain.
- Once added, a block cannot be modified.

---

2. Block Header

- The block header is used to identify a specific block in the blockchain.
- It contains important metadata required for mining and verification.
- Miners repeatedly hash the block header while changing the nonce.
- The block header plays a key role in block validation and security.

---

3. Previous Block Hash

- The previous block hash is a reference to the hash of the parent block.
- It connects the current block to the previous block in the chain.
- This linking ensures immutability, as changing one block affects all following blocks.
- It is the main reason blockchain data is tamper-evident.

---

4. Timestamp

- A timestamp records the date and time when a block is created.
- It helps verify when transactions were added to the blockchain.
- Timestamps provide chronological ordering of blocks.
- They also support auditability and transparency.

---

5. Nonce

- A nonce is a number used only once during block creation.
- It is a critical part of the Proof of Work mechanism.
- Miners continuously change the nonce to find a valid block hash.
- When a valid nonce is found, the block is accepted by the network.

---

6. Merkle Root

- The Merkle root is a single hash representing all transactions in a block.
- It is generated using a Merkle Tree, where transactions are hashed and combined.
- Any change in a transaction changes the Merkle root.
- This allows efficient and secure transaction verification.

Genesis Block in Blockchain

- The Genesis Block is the first block in a blockchain.
- It does not reference any previous block.
- It is hardcoded into the blockchain protocol.
- All other blocks trace their origin back to the Genesis Block.

Key Characteristics of Genesis Block

1. Unique Identifier

- The Genesis Block has a unique block number (usually 0).
- It marks the starting point of the blockchain.

2. No Previous Block Reference

- Since it is the first block, there is no previous block hash.
- The previous hash field contains a placeholder value (e.g., 0000).

3. Coinbase Transaction

- The first transaction in the Genesis Block is called the coinbase transaction.
- It creates new cryptocurrency units.
- It may include a message or timestamp information.

4. Establishes the Blockchain

- The Genesis Block initializes the blockchain.
- All subsequent blocks are linked to it.
- It forms the foundation of the immutable chain.

5. Hardcoded Information

- It may contain hardcoded data such as timestamp or creator message.
- This data is considered significant by developers.

Why Genesis Block is Needed

- It initializes the blockchain network.
- It establishes consensus on the initial state of the system.
- It provides a fixed and trusted starting point.
- It ensures traceability and integrity of all future blocks.
- It incentivizes early participation by setting initial parameters.

Significance of Genesis Block

- Acts as the starting point of the blockchain.
- Stores critical initial network information.
- Serves as a reference for all future blocks.
- Makes the blockchain immutable and secure.
- Has historical and symbolic importance in blockchain technology.

The data structure of blockchain consists of blocks linked using cryptographic hashes. Each block contains a header, previous block hash, timestamp, nonce, and Merkle root. The Genesis Block is the first block of the blockchain and serves as the foundation for the entire network.

UNIT 3 - Essentials of Blockchain

3.1 Consensus Mechanisms in Blockchain

Consensus = a general agreement

Definition

A consensus mechanism is a protocol used in blockchain to achieve agreement among distributed nodes on the validity of transactions and the state of the ledger.

In a decentralized blockchain network, there is no central authority to verify transactions. Consensus mechanisms ensure that all nodes agree on a single version of the blockchain, preventing fraud, and maintaining network security and trust.

Types of Consensus Mechanisms

---

1. Proof of Work (PoW)

Proof of Work is a consensus mechanism in which miners solve complex mathematical problems to validate transactions and create new blocks.

Miners compete to find a valid hash, and the first one to solve the problem adds the block to the blockchain and receives a reward.
It provides high security but requires large computational power and energy consumption.

---

2. Proof of Stake (PoS)

Proof of Stake selects validators based on the amount of cryptocurrency they hold and are willing to “stake” in the network.

Validators are chosen to create new blocks according to their stake, reducing the need for heavy computation.
It is more energy-efficient and faster compared to Proof of Work.

---

3. Delegated Proof of Stake (DPoS)

Delegated Proof of Stake is a variation of PoS where users vote to elect a small group of trusted delegates to validate transactions.

These delegates create and validate blocks on behalf of the network.
It improves speed and efficiency but reduces decentralization due to limited validators.

---

4. Practical Byzantine Fault Tolerance (PBFT)

PBFT is a consensus mechanism designed to tolerate faulty or malicious nodes in a distributed system.

Nodes communicate with each other and reach agreement through multiple rounds of verification.
It provides fast and reliable consensus but is mainly suitable for permissioned (private) blockchains.

---

5. Proof of Capacity (PoC)

Proof of Capacity uses available hard disk space instead of computational power to mine blocks.

Miners store precomputed data (plots) on their storage devices and use it to find valid blocks.
It consumes less energy compared to PoW and utilizes storage resources efficiently.

---

6. Proof of Authority (PoA)

Proof of Authority relies on a limited number of approved validators whose identities are known and trusted.

Validators are responsible for generating new blocks based on their reputation.
It offers high speed and efficiency but is less decentralized due to reliance on trusted authorities.

Consensus	Advantages	Disadvantages
PoW	Highly secure, well-tested, decentralized	High energy consumption, slow, expensive hardware
PoS	Energy efficient, faster, less hardware needed	Wealth concentration risk, less proven than PoW
DPoS	Very fast, efficient, scalable	Less decentralization, depends on voting system
PBFT	Fast finality, low energy use, fault tolerant	Not scalable for large networks, complex communication
PoC	Low energy use, uses storage instead of CPU	Requires large disk space, less popular
PoA	High speed, efficient, low cost	Centralized, depends on trusted validators

Feature	PoW	PoS	DPoS	PBFT	PoC	PoA
Basis of Selection	Computational power	Stake (coins held)	Voting for delegates	Node agreement	Storage capacity	Authority/identity
Energy Consumption	Very high	Low	Low	Very low	Low	Very low
Speed	Slow	Faster than PoW	Fast	Very fast	Moderate	Very fast
Decentralization	High	Moderate	Low	Low	Moderate	Low
Security	Very high	High	Moderate	High (small network)	Moderate	Moderate
Resource Requirement	CPU/GPU power	Cryptocurrency stake	Voting system	Communication among nodes	Disk space	Trusted validators
Usage Type	Public blockchain	Public blockchain	Public blockchain	Private blockchain	Public blockchain	Private/consortium

3.2 Confirmation and Finality

Confirmation

Confirmation refers to the number of blocks added after a transaction's block in the blockchain.

Explanation:
When a transaction is included in a block, it gets its first confirmation.
Each new block added increases the confirmation count, making the transaction more secure.
More confirmations reduce the chances of reversal due to forks or attacks.

Confirmation = Number of blocks added after a transaction's block
More confirmations → more trust → more security

When a transaction is made in blockchain, it is **not immediately final**.
First:

- It gets included in a **block**
- That block gets added to the blockchain

👉 This is called **1 confirmation**

Finality

Definition:
Finality is the assurance that a transaction cannot be reversed or altered once confirmed.

Explanation:
In blockchain, finality means the transaction is permanently recorded.
In Proof of Work systems, finality is probabilistic, meaning it becomes more secure over time but is not instant.
Some systems provide immediate finality, where transactions cannot be changed once confirmed.

Limits of Proof of Work

1. Probabilistic Finality

Transactions are not instantly final and require multiple confirmations to become secure.
There is always a small possibility of chain reorganization and transaction reversal.

---

2. High Energy Consumption

PoW requires significant computational power, leading to high electricity usage.
This makes it inefficient and environmentally costly.

---

3. Slow Transaction Speed

Block creation takes time, resulting in slower transaction processing.
Not suitable for high-speed applications like real-time payments.

---

4. 51% Attack Risk

If a single entity gains majority computational power, it can manipulate the blockchain.
This can lead to double spending and transaction control.

---

5. Scalability Issues

Limited block size and block time reduce the number of transactions per second.
This affects the scalability of the network.

Alternatives of Proof of Work

1. Proof of Stake (PoS)

Validators are selected based on their stake in the network.
It reduces energy consumption and provides faster transaction processing.

---

2. Delegated Proof of Stake (DPoS)

Users vote for delegates who validate transactions.
It increases speed and efficiency but reduces decentralization.

3. Practical Byzantine Fault Tolerance (PBFT)

Nodes reach agreement through communication and voting.
Provides fast and deterministic finality, mainly used in private blockchains.

---

4. Proof of Authority (PoA)

Trusted validators are responsible for block creation.
Offers high speed and efficiency but depends on known authorities.

---

5. Proof of Capacity (PoC)

Uses storage space instead of computational power.
Consumes less energy and is more efficient than PoW.

3.3 Block Rewards, Miners and Difficulty under Competition

Miners

Definition:
Miners are participants (nodes) in a blockchain network who validate transactions and add new blocks to the blockchain.

Explanation:
Miners collect pending transactions and attempt to create a valid block by solving cryptographic puzzles.
They compete with other miners to be the first to add the block to the chain.
Mining ensures security, verification of transactions, and proper functioning of the blockchain network.

Block Rewards

Definition:
Block reward is the incentive given to a miner for successfully adding a new block to the blockchain.

Explanation:
It consists of newly generated cryptocurrency (block subsidy) and transaction fees included in the block.
Rewards motivate miners to participate in the network and maintain its security.
Over time, block rewards may decrease, making transaction fees more important.

Difficulty

Definition:
Difficulty is a measure of how hard it is to solve the cryptographic puzzle required to mine a block.

Explanation:
The network adjusts difficulty to maintain a consistent block generation time.
If blocks are mined too quickly, difficulty increases; if too slowly, it decreases.
Higher difficulty means more computational effort is required to find a valid hash.

Difficulty under Competition

Definition:
Difficulty under competition refers to the adjustment of mining difficulty based on the number of miners and total computational power in the network.

Explanation:
As more miners join, total hashing power increases, causing blocks to be mined faster.
The system responds by increasing difficulty to maintain stability in block time.
This creates a competitive environment where miners require more resources to succeed.

3.4 Forks and Consensus Chain

Forks

Definition:
A fork is a situation in blockchain where the chain splits into two or more possible paths due to disagreement among nodes.

Explanation:
Forks usually occur when two miners generate blocks at the same time or when there is a change in protocol rules.
This creates multiple versions of the blockchain temporarily or permanently.
Forks can be resolved automatically (temporary forks) or may lead to separate blockchains (hard forks).

Types of Forks

---

1. Temporary Fork (Soft Fork / Accidental Fork)

Definition:
A temporary fork is a short-term split in the blockchain caused by simultaneous block creation.

Explanation:
Different nodes accept different blocks at the same height, creating parallel chains.
The network later resolves this by selecting the chain with more work (longest chain).
The discarded blocks are called orphan blocks.

2. Hard Fork

Definition:
A hard fork is a permanent split in the blockchain due to changes in protocol rules that are not backward compatible.

Explanation:
Nodes that follow new rules create a separate chain from those following old rules.
This results in two independent blockchains running simultaneously.
It is often used to introduce major upgrades or changes.

Consensus Chain

Definition:
The consensus chain is the blockchain that is accepted as valid by the majority of nodes based on consensus rules.

Explanation:
In case of forks, multiple chains may exist temporarily.
The network selects the chain with the highest cumulative work (usually the longest chain) as the valid one.
This ensures that all nodes eventually agree on a single version of the blockchain.

Point	Hard Fork	Soft Fork
Definition	A hard fork is a permanent change in blockchain rules that is not backward compatible.	A soft fork is a temporary or backward-compatible change in blockchain rules.
Compatibility	Not compatible with old nodes; old nodes cannot validate new blocks.	Compatible with old nodes; they can still accept new blocks.
Result	Creates two separate blockchains if all nodes don’t upgrade.	Does not create a separate chain if majority follows new rules.
Node Requirement	Requires all nodes to upgrade to new version.	Only majority of nodes need to upgrade.
Example Use	Used for major changes or upgrades in protocol.	Used for minor improvements or rule tightening.

3.5 Sybil Attacks and 51% Attack

Sybil Attack

Definition:
A Sybil attack is a security threat in which a single entity creates multiple fake identities (nodes) in a network to gain influence or control.

Explanation:
The attacker pretends to be many different nodes and tries to manipulate the network by increasing its influence.
It can affect voting, communication, and decision-making in distributed systems.
Blockchain systems like Proof of Work reduce this risk by requiring computational power instead of just multiple identities.

51% Attack

Definition:
A 51% attack occurs when a single miner or group controls more than 50% of the total computational power of the blockchain network.

Explanation:
With majority power, the attacker can control block creation and manipulate the blockchain.
They can reverse transactions, perform double spending, and prevent other transactions from being confirmed.
However, they cannot create new coins arbitrarily or directly steal from other wallets.

Aspect	Sybil Attack	51% Attack
Method	Multiple fake identities	Majority computational power
Target	Network influence	Blockchain control
Requirement	Easy to create nodes	Requires huge resources
Impact	Disrupts communication/voting	Enables double spending and control

UNIT 4 - Conceptualization of Blockchain as cryptocurrency

4.1 Bitcoin: Merkle Tree and Bitcoin

Merkle Tree

Definition:
A Merkle tree is a binary tree structure used to efficiently store and verify large sets of transaction data using hashes.

Explanation:
Each leaf node represents a transaction hash, and parent nodes store combined hashes of their children.
The top hash is called the **Merkle Root**, stored in the block header.
It allows quick verification without downloading all transactions.

Bitcoin

Definition:
Bitcoin is a decentralized digital currency that operates on a peer-to-peer blockchain network without a central authority.

Explanation:
Transactions are verified by miners and recorded in blocks.
It uses Proof of Work to maintain security and consensus.
All transactions are publicly stored on a distributed ledger.

4.2 Bitcoin and Eventual Consistency, Byzantine Fault Tolerance

Eventual Consistency

Definition:
Eventual consistency means all nodes in a distributed system will become consistent over time.

Explanation:
In Bitcoin, different nodes may temporarily have different versions of the blockchain.
Due to propagation delay and forks, inconsistency can occur briefly.
Eventually, all nodes agree on the longest valid chain.

Byzantine Fault Tolerance (BFT)

Definition:
Byzantine Fault Tolerance is the ability of a system to function correctly even when some nodes act maliciously or fail.

Explanation:
Bitcoin handles Byzantine faults using Proof of Work.

Honest nodes follow consensus rules and reject invalid blocks.
As long as the majority is honest, the network remains secure.

4.3 Bitcoin and Secure Hashing, Block Size, Mining

Secure Hashing

Definition:
Secure hashing is a process of converting data into a fixed-size string using a cryptographic hash function.

Explanation:
Bitcoin uses SHA-256 hashing for blocks and transactions.
It ensures data integrity and makes tampering extremely difficult.
Even a small change in input produces a completely different hash.

Bitcoin Block Size

Definition:
Block size refers to the maximum amount of data that a Bitcoin block can hold.

Explanation:
Originally limited to 1 MB, affecting number of transactions per block.
Smaller block size leads to limited throughput and congestion.
Larger blocks can increase speed but affect decentralization.

Bitcoin Mining

Definition:
Bitcoin mining is the process of validating transactions and adding new blocks by solving cryptographic puzzles.

Explanation:
Miners compete to find a valid hash using computational power.
The first miner to solve it adds the block and gets rewards.
Mining secures the network and maintains consensus.

4.4 Proof of Work and Bitcoin Scripting

Proof of Work (PoW)

Definition:
Proof of Work is a consensus mechanism where miners solve complex mathematical problems to validate transactions.

Explanation:
It requires computational effort to add new blocks.

Ensures security by making attacks costly.
Used by Bitcoin to achieve decentralized consensus.

Bitcoin Scripting

Definition:
Bitcoin scripting is a simple programming language used to define transaction conditions.

Explanation:
It specifies rules like who can spend the coins.
Scripts are not Turing complete, making them secure and limited.
Used for features like multi-signature transactions.

4.5 Blockchain Collaborative Implementations

Hyperledger

Definition

Hyperledger is an open-source blockchain framework designed for enterprise and business applications, mainly supporting permissioned (private) networks.

Need of Hyperledger

- Public blockchains lack privacy → business data needs confidentiality
- Enterprises require controlled access and identity management
- Need for high performance and scalability
- Requirement of trusted participants instead of anonymous users

---

Technical Layers of Hyperledger

1. Application Layer

This is the top layer where business applications interact with blockchain.
It includes user interfaces, APIs, and client applications.

---

2. Smart Contract Layer (Chaincode)

Contains business logic written as smart contracts.
Executes rules and processes transactions automatically.

---

3. Consensus Layer

Responsible for agreement among nodes on transaction validity.
Ensures consistency and correctness of data.

---

4. Data Layer

Stores blockchain data such as transactions and ledger state.
Includes databases and world state storage.

---

5. Network Layer

Handles communication between nodes in the network.
Ensures secure and efficient data exchange.

Advantages of Hyperledger

- High privacy and permission control
- Better scalability and performance
- Modular architecture (flexible design)
- Suitable for enterprise use
- No need for cryptocurrency

Disadvantages of Hyperledger

- Less decentralized compared to public blockchains
- Complex setup and management
- Limited transparency
- Requires trust among participants

Corda

Definition

Corda is a distributed ledger platform designed mainly for financial and business transactions with high privacy.

Key Points (Understanding)

- Not a traditional blockchain (no global broadcast of data)
- Data is shared only between involved parties
- Focus on legal agreements and real-world contracts

Advantages

- High privacy (data shared only with relevant parties)
- Efficient and scalable
- Designed for regulated industries
- Strong integration with legal systems

Disadvantages

- Less decentralization
- Limited use outside enterprise
- Smaller ecosystem compared to Ethereum

ERC-20

Definition

ERC-20 is a standard protocol used to create and manage tokens on the Ethereum blockchain.

ERC-20 Components (Functions)

- **totalSupply()** – total number of tokens
- **balanceOf()** – check token balance
- **transfer()** – transfer tokens
- **approve()** – allow others to spend tokens
- **transferFrom()** – transfer on behalf of owner
- **allowance()** – check approved amount

Advantages of ERC-20

- Standardization (easy integration)
- Widely supported by wallets and exchanges
- Easy token creation
- Supports smart contract functionality

Disadvantages of ERC-20

- Transaction fees (gas cost)
- Network congestion
- Limited flexibility
- Smart contract bugs can cause losses

Applications of ERC-20

- Cryptocurrency tokens
- Utility tokens in apps
- Fundraising (ICO)
- DeFi platforms
- Reward and loyalty systems

Token

Definition

A token is a digital asset created on a blockchain that represents value, ownership, or access rights.

Types of Tokens

- Utility tokens (access to services)
- Security tokens (represent assets)
- Governance tokens (voting rights)

Advantages of Tokens

- Easy transfer and trade
- Programmable and flexible
- Supports decentralized systems
- Global accessibility

Disadvantages of Tokens

- Regulatory issues
- Security risks
- Market volatility
- Dependency on blockchain platform

Applications of Tokens

- Digital payments
- Asset representation
- Voting systems
- Gaming and NFTs

Hyperledger

Definition:
Hyperledger is an open-source blockchain platform designed for enterprise applications.

Explanation:
It supports permissioned networks where participants are known.
Focuses on privacy, scalability, and modular architecture.
Used in industries like finance and supply chain.

Corda

Definition:
Corda is a distributed ledger platform designed for business transactions.

Explanation:
It is not a traditional blockchain; data is shared only between involved parties.
Focuses on privacy and legal agreements.
Widely used in banking and financial services.

ERC-20

Definition:
ERC-20 is a standard for creating tokens on the Ethereum blockchain.

Explanation:
It defines rules for token transfer and balance tracking.
Ensures compatibility between different applications and wallets.
Used for cryptocurrencies and utility tokens.

Token

Definition:
A token is a digital asset created on a blockchain representing value or utility.

Explanation:
Tokens can represent currency, assets, or access rights.
They are created using smart contracts.
Widely used in decentralized applications and fundraising.

UNIT 5 - Decentralization using Blockchain

5.1 Blockchain and Full Decentralization

Decentralization

Definition

Decentralization is a system in which control, data, and decision-making are distributed among multiple nodes instead of being controlled by a single central authority.

Simple Explanation

In blockchain, no single entity controls the network.
All nodes maintain a copy of data and participate in validation.
This removes dependency on intermediaries like banks or servers.

Types of Decentralization

1. Architectural Decentralization

Refers to how many physical nodes are present in the system.
More nodes → more distributed system → less chance of failure.

---

2. Political Decentralization

Refers to how control is distributed among participants.
If no single entity controls decision-making, the system is politically decentralized.

---

3. Logical Decentralization

Refers to how the system behaves as a whole.
Even if many nodes exist, the system may act as one unit (like blockchain).

Advantages of Decentralization

- No single point of failure
- Increased security and reliability
- Reduces need for intermediaries
- Greater transparency
- Resistant to censorship

Disadvantages of Decentralization

- Slower performance
- Difficult to manage and govern
- High resource usage
- Scalability issues
- Decision-making can be complex

Smart Contract

Basic definition

A smart contract is a self-executing program stored on a blockchain that automatically runs when predefined conditions are met

---

Simple understanding

Think of it like:

“If this happens → then automatically do this”

Example:

- If payment is received → transfer ownership
- No need for middleman

Smart Contract Working

1. Identify Agreement

Parties decide what they want to do and agree on the terms.
This can include payments, services, or exchange of assets.

---

2. Set Conditions

Conditions are defined for when the contract should run.
The contract works only when these conditions are met.

---

3. Code Business Logic

The agreement is written in code with clear rules.
This code tells the system what action to take.

---

4. Encryption and Blockchain

Data is secured using encryption and stored on blockchain.
This ensures safety and prevents tampering.

---

5. Execution and Processing

When conditions are satisfied, the contract runs automatically.
The transaction is verified and completed by the network.

---

6. Network Updates

All nodes update their records after execution.
The data becomes permanent and cannot be changed.

Features of Smart Contracts

- **Automation** – Executes automatically without human involvement
- **Immutability** – Cannot be changed once deployed
- **Transparency** – Code is visible and verifiable
- **Security** – Uses cryptographic protection
- **Trustless** – No need for third-party trust

Capabilities of Smart Contracts

- Transfer of digital assets
- Execution of agreements automatically
- Managing ownership and rights
- Enforcing rules and conditions
- Supporting decentralized applications (DApps)

Types of Smart Contracts

1. Smart Legal Contracts

Legally enforceable contracts written in code form.
Used in agreements like insurance or property deals.

---

2. Decentralized Autonomous Organizations (DAO)

Organizations run by smart contracts without central authority.
Decisions are made through voting mechanisms.

---

3. Application Logic Contracts (ALC)

Used in blockchain applications to control business logic.
They handle interactions between users and blockchain.

Advantages of Smart Contracts

- Eliminates intermediaries
- Faster execution
- Reduces cost
- Increases accuracy
- Secure and tamper-proof

Challenges of Smart Contracts

- Difficult to modify once deployed
- Bugs in code can cause loss
- Requires technical knowledge
- Legal status is not fully defined
- Not suitable for all real-world cases

OPTIONAL

1. What problem did Blockchain try to solve?

Before blockchain, **all digital records were stored and controlled by a central authority**.

Traditional system (Centralized):

- Bank controls money records
- University controls student records
- Company controls employee data

Problems with this system:

1. **Single point of failure**
   If the central server fails → system stops
2. **Trust issue**
   We must blindly trust the central authority
3. **Data manipulation**
   Records can be changed secretly
4. **Lack of transparency**

👉 Blockchain was created to **remove blind trust** and **remove central control**.

---

2. What is a Ledger?

A **ledger** is simply:

A record book where transactions are written.

Examples:

- Bank passbook
- Accounting book
- Excel sheet of transactions

---

3. What is a Distributed Ledger?

Definition (simple):

A **distributed ledger** is a ledger that is:

- **Shared**
- **Copied**
- **Synchronized**
  across **many computers (nodes)** instead of one central server.

Key idea:

Every participant has the **same copy** of the ledger.

Example:

- 10 computers in a network
- All 10 have **identical transaction records**
- If one fails, others still have data

---

4. What is Blockchain? (Core Definition)

Simple definition:

Blockchain is a type of distributed ledger where data is stored in blocks, and blocks are connected using cryptography.

Standard definition (exam-ready):

Blockchain is a **decentralized, distributed, immutable digital ledger** that records transactions in blocks which are linked together using cryptographic hashes.

---

5. Why the name "Blockchain"?

Because:

- Data is stored in **Blocks**
- Blocks are connected in a **Chain**

So:
Block + Chain = Blockchain

---

6. Structure of a Block

Each **block** contains:

1 Block Header

- **Block number**
- **Timestamp** (date & time)
- **Previous block hash**
- **Current block hash**
- **Nonce** (used in mining)

2 Transaction Data

- List of transactions

3️⃣ Hash

- A unique digital fingerprint of the block

👉 If even **one letter** in the block changes → hash changes.

---

7. What is Hash? (Simple but powerful)

Hash means:

A fixed-length encrypted output generated from input data.

Properties:

- Same input → same hash
- Small change in input → completely different hash
- Cannot reverse hash to get original data

Why hash is important?

- Ensures **data integrity**
- Makes blockchain **tamper-proof**

---

8. How Blocks are Linked (Chain Formation)

- Every block stores the **hash of previous block**
- If someone tries to change one block:
    - Its hash changes
    - Next block becomes invalid
    - Entire chain breaks

👉 This makes blockchain **immutable**.

---

9. Working of Blockchain (Step-by-step Flow)

Let's understand with a simple transaction.

Step 1: Transaction request

A user requests a transaction
(e.g., A sends money to B)

Step 2: Broadcast

Transaction is broadcast to all nodes in the network

Step 3: Verification

Nodes verify:

- Valid sender?
- Sufficient balance?
- Correct digital signature?

Step 4: Block creation

Verified transactions are grouped into a **block**

Step 5: Consensus

Network agrees on the block using a **consensus mechanism**

Step 6: Block added to chain

- Block is added permanently
- Ledger is updated on all nodes

---

10. Key Components of Blockchain

1 Nodes

- Computers participating in blockchain network
- Store ledger copies

2 Blocks

- Data containers

3 Chain

- Connection of blocks via hashes

4 Distributed Ledger

- Shared transaction record

5 Consensus Mechanism

- Method to agree on valid transactions

6 Cryptography

- Hashing
- Digital signatures
- Public & private keys

---

11. Consensus Mechanism (Basic idea only)

Consensus means:

How all nodes agree that a transaction is valid.

Without consensus:

- Fake transactions possible
- No trust

Examples (just names for now):

- Proof of Work (PoW)
- Proof of Stake (PoS)

(We'll study them later in detail.)

---

12. Key Characteristics of Blockchain

1️⃣ Decentralization

- No central authority

2️⃣ Transparency

- All transactions visible to participants

3️⃣ Immutability

- Data cannot be changed once added

4️⃣ Security

- Cryptography + hashing

5️⃣ Trustless system

- No need to trust intermediaries